

CS162
Operating Systems and
Systems Programming
Lecture 11

Scheduling 2:
Fair Sharing, Lottery Scheduling, EEVDF

October 8th, 2025

Professor Ion Stoica & Matei Zaharia

<http://cs162.eecs.Berkeley.edu>

Scheduling Fairness

- Two identical processes should get
 - the “same” service, experience the “same” performance
- Work-conserving scheduler: does not leave the CPU idle when there is work to do
- Note: fairness avoids starvation
- Example: round-robin
- Next:
 - Fair sharing / Fair queueing / Max-min fairness)
 - Originated in networking
 - Later applied to CPU scheduling and other resources (lottery scheduling)

Analysis and Simulation of a Fair Queueing Algorithm



Alan Demers
Srinivasan Keshav
Scott Shenker
Xerox PARC
3333 Coyote Hill Road
Palo Alto, CA 94304

Abstract

We discuss gateway queueing algorithms and their role in controlling congestion in datagram networks. A fair queueing algorithm, based on an earlier suggestion by Nagle, is proposed. Analysis and simulations are used to compare this algorithm to other congestion control schemes. We find that fair queueing provides several important advantages over the usual first-come-first-serve queueing algorithm: fair allocation of bandwidth, lower delay for sources using less than their full share of bandwidth, and protection from ill-behaved sources.

often ignored, makes queueing algorithms a crucial component in effective congestion control.

Queueing algorithms can be thought of as allocating three nearly independent quantities: bandwidth (which packets get transmitted), promptness (when do those packets get transmitted), and buffer space (which packets are discarded by the gateway). Currently, the most common queueing algorithm is first-come-first-serve (FCFS). FCFS queueing essentially relegates all congestion control to the sources, since the order of arrival completely determines the bandwidth, promptness, and buffer space allocations. Thus, FCFS inextricably intertwines these

SIGCOMM '89

Lottery Scheduling: Flexible Proportional-Share Resource Management

Carl A. Waldspurger * William E. Weihl *
MIT Laboratory for Computer Science
Cambridge, MA 02139 USA

Abstract

This paper presents *lottery scheduling*, a novel randomized resource allocation mechanism. Lottery scheduling provides efficient, responsive control over the relative execution rates of computations. Such control is beyond the capabilities of conventional schedulers, and is desirable in systems that service requests of varying importance, such as databases, media-based applications, and networks. Lottery scheduling also supports modular resource management by enabling concurrent modules to insulate their resource allocation policies from one another. A *currency* abstraction is introduced to flexibly name, share, and protect resource rights. We also show that lottery scheduling can be generalized to manage many diverse resources, such as I/O bandwidth, memory, and access to locks. We have implemented a prototype lottery scheduler for the Mach 3.0 microkernel, and found that it provides flexible and responsive control over the relative execution rates of a wide range of applications. The overhead imposed by our unoptimized prototype is comparable to that of the standard Mach timesharing policy.

to rapidly focus available resources on tasks that are currently important [Dui90].

Few general-purpose schemes even come close to supporting flexible, responsive control over service rates. Those that do exist generally rely upon a simple notion of *priority* that does not provide the encapsulation and modularity properties required for the engineering of large software systems. In fact, with the exception of hard real-time systems, it has been observed that the assignment of priorities and dynamic priority adjustment schemes are often ad-hoc [Dei90]. Even popular priority-based schemes for CPU allocation such as *decay-usage scheduling* are poorly understood, despite the fact that they are employed by numerous operating systems, including Unix [Hel93].

Existing *fair share* schedulers [Hen84, Kay88] and *microeconomic* schedulers [Fer88, Wal92] successfully address some of the problems with absolute priority schemes. However, the assumptions and overheads associated with these systems limit them to relatively coarse control over

OSDI '94

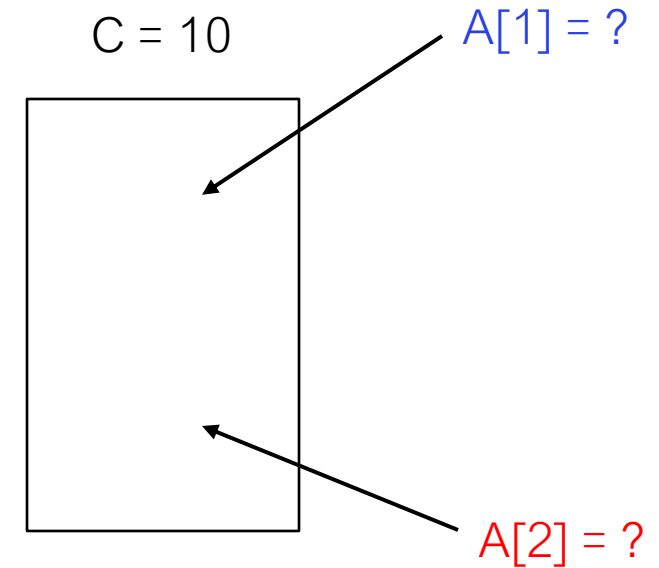
Fair allocation: Example 1

Resource with capacity, $C = 10$

Two jobs with demands $D[1] = 8$, $D[2] = 6$

How much should each job get?

- $A[1] = ?$
- $A[2] = ?$



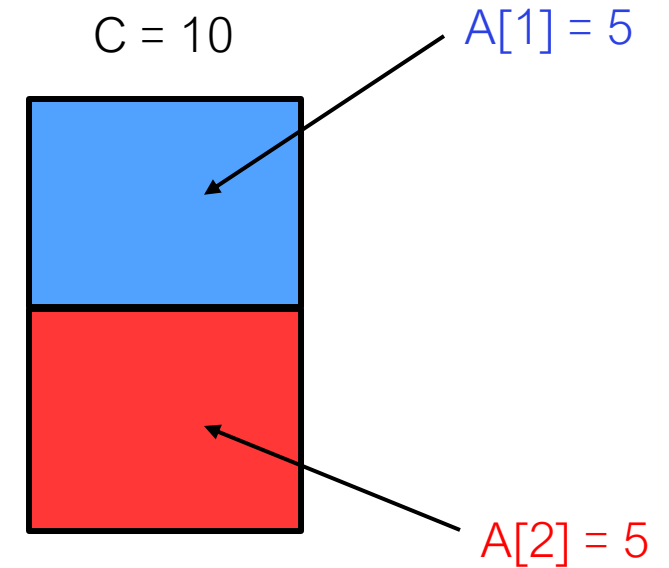
Fair allocation: Example 1

Resource with capacity, $C = 10$

Two jobs with demands $D[1] = 8$, $D[2] = 6$

How much should each job get?

- $A[1] = 5$
- $A[2] = 5$



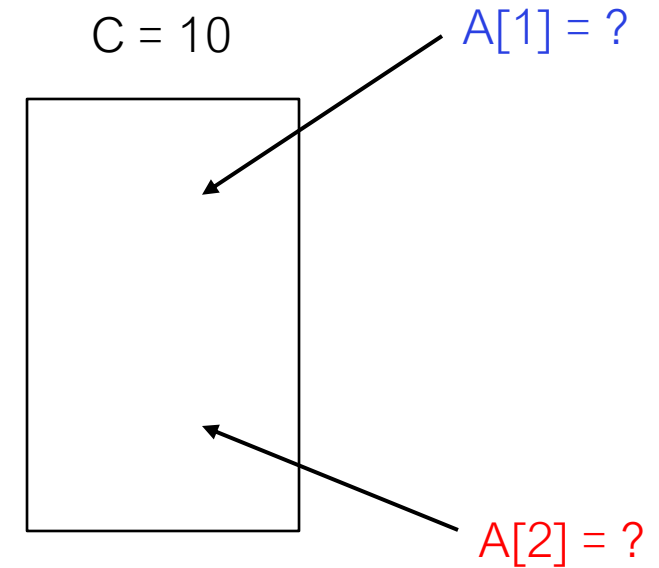
Fair allocation: Example 2

Resource with capacity, $C = 10$

Two jobs with demands $D[1] = 8$, $D[2] = 4$

How much should each job get?

- $A[1] = ?$
- $A[2] = ?$



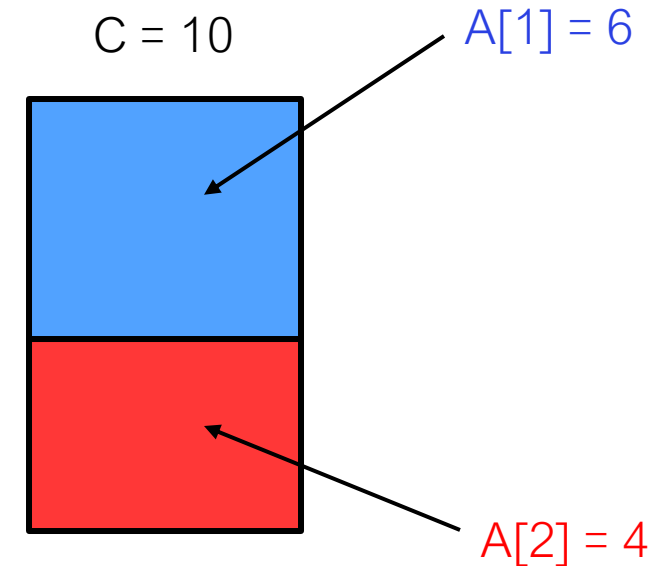
Fair allocation: Example 2

Resource with capacity, $C = 10$

Two jobs with demands $D[1] = 8$, $D[2] = 4$

How much should each job get?

- $A[1] = 6$
- $A[2] = 4$



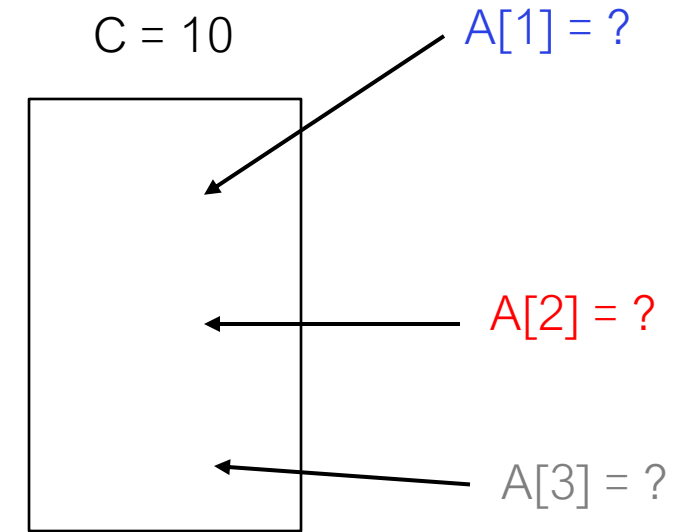
Fair allocation: Example 3

Resource with capacity, $C = 10$

Three jobs with demands $D[1] = 8$, $D[2] = 6$, $D[3] = 2$

How much should each job get?

- $A[1] = ?$
- $A[2] = ?$
- $A[3] = ?$



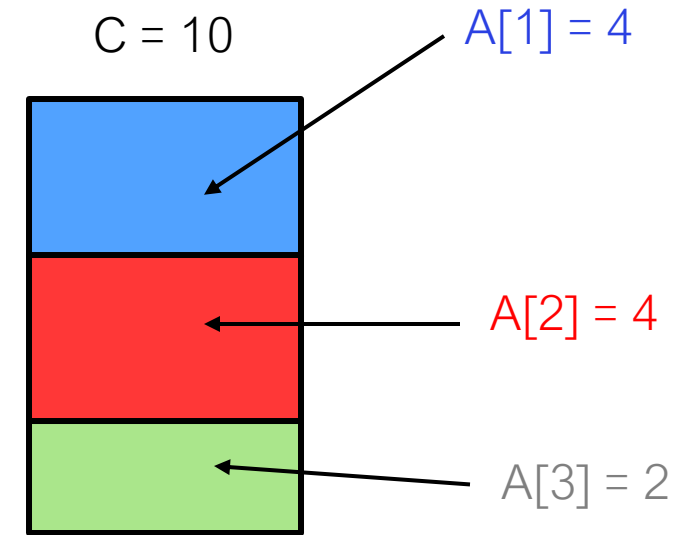
Fair allocation: Example 3

Resource with capacity, $C = 10$

Three jobs with demands $D[1] = 8$, $D[2] = 6$, $D[3] = 2$

How much should each job get?

- $A[1] = ?$
- $A[2] = ?$
- $A[3] = ?$



Max-Min Fairness

```
C # resource capacity
S = {J1, J2, ..., Jn} # set of n jobs
D[n] # demand vector; D[i]: demand of job i
A[n] # allocation vector; A[i], allocation of job i
while (S != empty):
    f = C/|S| # fair share
    b = true # no job in S has demand < f
    for every Ji in S:
        if D[i] < f
            b = false
            A[i] = D[i]
            C = C - D[i]
            S = S \ {Ji}
    if b:
        for every Ji in S:
            A[i] = f
            S = S \ {Ji}
```

Max-Min Fairness: Running Example

$C = 10$ # resource capacity

$S = \{J1, J2, J3\}$ # set of N jobs

$D = [8, 6, 2]$ # demand vector; $D[i]$: demand of job i

$A = [0, 0, 0]$ # allocation vector; $A[i]$, allocation of job i

while ($S \neq \text{empty}$):

$f = C/|S|$ # fair share

$b = \text{true}$ # no job in S has demand $< f$

 for every j in S :

 if $D[i] \leq f$

$b = \text{false}$

$A[i] = D[i]$

$C = C - D[i]$

$S = S \setminus \{j\}$

 if b :

 for every j in S :

$A[i] = f$

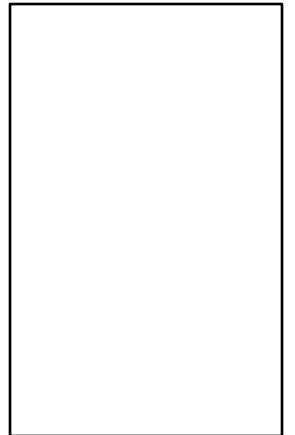
$S = S \setminus \{j\}$

$C = 10$

$S = \{J1, J2, J3\}$

$D[1] = 8, D[2] = 6, D[3] = 2$

10



Max-Min Fairness: Running Example

$C = 10$ # resource capacity

$S = \{J1, J2, J3\}$ # set of N jobs

$D = [8, 6, 2]$ # demand vector; $D[i]$: demand of job i

$A = [0, 0, 0]$ # allocation vector; $A[i]$, allocation of job i

while ($S \neq \text{empty}$):

$f = C/|S|$ # fair share

$b = \text{true}$ # no job in S has demand $< f$

for every j in S :

if $D[i] \leq f$

$b = \text{false}$

$A[i] = D[i]$

$C = C - D[i]$

$S = S \setminus \{j\}$

if b :

for every j in S :

$A[i] = f$

$S = S \setminus \{j\}$

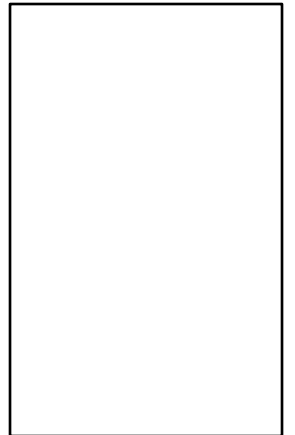
$C = 10$

$S = \{J1, J2, J3\}$

$D[1] = 8, D[2] = 6, D[3] = 2$

$f = 10/3$

10



Max-Min Fairness: Running Example

```
C = 10 # resource capacity
S = {J1, J2, J3} # set of N jobs
D = [8, 6, 2] # demand vector; D[i]: demand of job i
A = [0, 0, 0] # allocation vector; A[i], allocation of job i
while (S != empty):
    f = C/|S| # fair share
    b = true # no job in S has demand < f
    for every j in S:
        if D[i] <= f
            b = false
            A[i] = D[i]
            C = C - D[i]
            S = S \ {j}
    if b:
        for every j in S:
            A[i] = f
            S = S \ {j}
```

C = 10

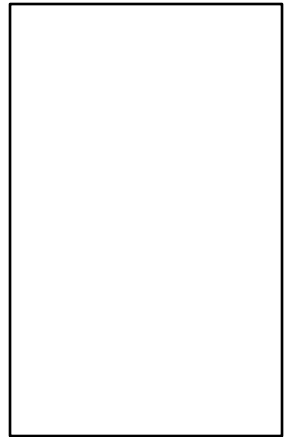
S = {J1, J2, J3}

D[1] = 8, D[2] = 6, D[3] = 2

f = 10/3

b = true

10



Max-Min Fairness: Running Example

$C = 10$ # resource capacity

$S = \{J1, J2, J3\}$ # set of N jobs

$D = [8, 6, 2]$ # demand vector; $D[i]$: demand of job i

$A = [0, 0, 0]$ # allocation vector; $A[i]$, allocation of job i

while ($S \neq \text{empty}$):

$f = C/|S|$ # fair share

$b = \text{true}$ # no job in S has demand $< f$

 for every j in S :

 if $D[i] \leq f$

$b = \text{false}$

$A[i] = D[i]$

$C = C - D[i]$

$S = S \setminus \{j\}$

 if b :

 for every j in S :

$A[i] = f$

$S = S \setminus \{j\}$

$C = 10$

$S = \{J1, J2, J3\}$

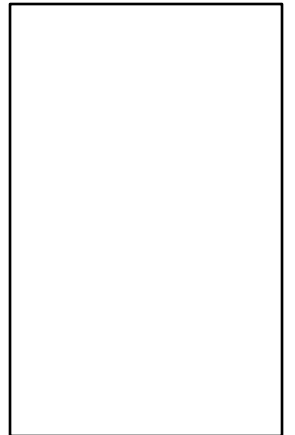
$D[1] = 8, D[2] = 6, D[3] = 2$

$f = 10/3$

$b = \text{true}$

$D[3] < 10/3 \rightarrow A[3] = D[3]$

10



Max-Min Fairness: Running Example

$C = 10$ # resource capacity

$S = \{J1, J2, J3\}$ # set of N jobs

$D = [8, 6, 2]$ # demand vector; $D[i]$: demand of job i

$A = [0, 0, 0]$ # allocation vector; $A[i]$, allocation of job i

while ($S \neq \text{empty}$):

$f = C/|S|$ # fair share

$b = \text{true}$ # no job in S has demand $< f$

 for every j in S :

 if $D[i] \leq f$

$b = \text{false}$

$A[i] = D[i]$

$C = C - D[i]$

$S = S \setminus \{j\}$

 if b :

 for every j in S :

$A[i] = f$

$S = S \setminus \{j\}$

$C = 10 - 2 = 8$

$S = \{J1, J2\}$

$D[1] = 8, D[2] = 6, D[3] = 2$

$f = 10/3$

$b = \text{false}$

$D[3] < 10/3 \rightarrow A[3] = D[3]$

10

$A[3] = 2$

Max-Min Fairness: Running Example

$C = 10$ # resource capacity

$S = \{J1, J2, J3\}$ # set of N jobs

$D = [8, 6, 2]$ # demand vector; $D[i]$: demand of job i

$A = [0, 0, 0]$ # allocation vector; $A[i]$, allocation of job i

while ($S \neq \text{empty}$):

$f = C/|S|$ # fair share

$b = \text{true}$ # no job in S has demand $< f$

for every j in S :

if $D[i] \leq f$

$b = \text{false}$

$A[i] = D[i]$

$C = C - D[i]$

$S = S \setminus \{j\}$

if b :

for every j in S :

$A[i] = f$

$S = S \setminus \{j\}$

$C = 8$

$S = \{J1, J2\}$

$D[1] = 8, D[2] = 6, D[3] = 2$

$f = 8/2 = 4$

$b = \text{true}$

10

$A[3] = 2$

Max-Min Fairness: Running Example

$C = 10$ # resource capacity

$S = \{J1, J2, J3\}$ # set of N jobs

$D = [8, 6, 2]$ # demand vector; $D[i]$: demand of job i

$A = [0, 0, 0]$ # allocation vector; $A[i]$, allocation of job i

while ($S \neq \text{empty}$):

$f = C/|S|$ # fair share

$b = \text{true}$ # no job in S has demand $< f$

 for every j in S :

 if $D[i] \leq f$

$b = \text{false}$

$A[i] = D[i]$

$C = C - D[i]$

$S = S \setminus \{j\}$

 if b :

 for every j in S :

$A[i] = f$

$S = S \setminus \{j\}$

$C = 8$

$S = \{J1, J2\}$

$D[1] = 8, D[2] = 6, D[3] = 2$

$f = 4$

$b = \text{true}$

$D[1] > f; D[2] > f$

10

$A[3] = 2$

Max-Min Fairness: Running Example

$C = 10$ # resource capacity

$S = \{J1, J2, J3\}$ # set of N jobs

$D = [8, 6, 2]$ # demand vector; $D[i]$: demand of job i

$A = [0, 0, 0]$ # allocation vector; $A[i]$, allocation of job i

while ($S \neq \text{empty}$):

$f = C/|S|$ # fair share

$b = \text{true}$ # no job in S has demand $< f$

 for every j in S :

 if $D[i] \leq f$

$b = \text{false}$

$A[i] = D[i]$

$C = C - D[i]$

$S = S \setminus \{j\}$

 if b :

 for every j in S :

$A[i] = f$

$S = S \setminus \{j\}$

$C = 8$

$S = \{ \}$

$D[1] = 8, D[2] = 6, D[3] = 2$

$f = 4$

$b = \text{true}$

$A[1] = 4$

$A[2] = 4$

10

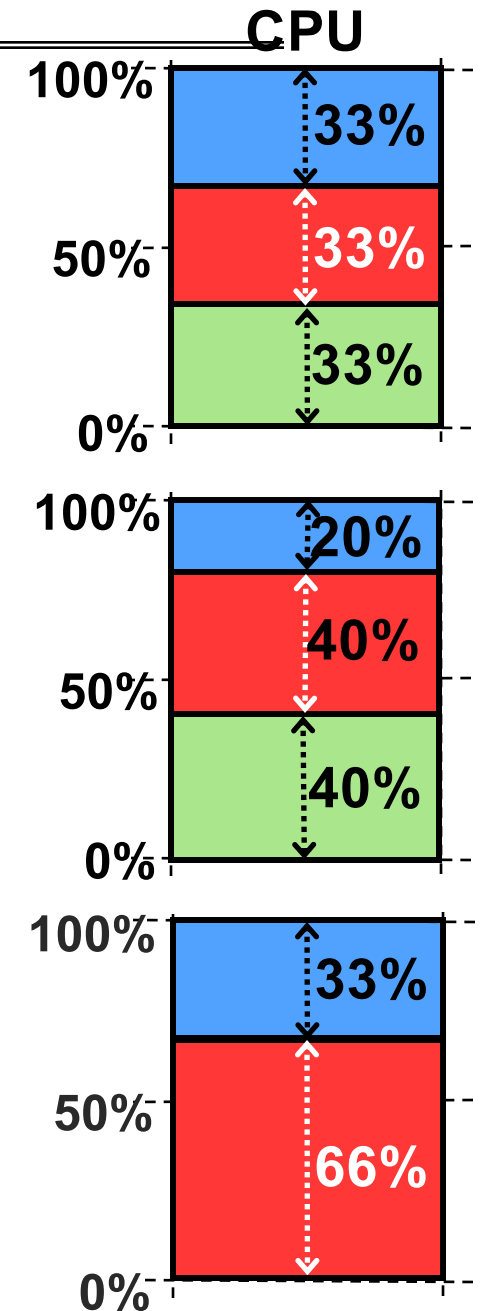
$A[1] = 4$

$A[2] = 4$

$A[3] = 2$

Single Resource: Fair Sharing

- n jobs want to share a resource (e.g. CPU)
 - Solution: give each $1/n$ of the shared resource
 - Assume $n = 3$
- Generalized by *max-min fairness*
 - Handles if a job wants less than its fair share
 - E.g. job 1 wants no more than 20%
- Generalized by *weighted max-min fairness*
 - Give weights to jobs according to importance
 - Job 1 gets weight 1, job 2 weight 2



Widely Used

- *OS:* proportional sharing, lottery, Linux's CFS, ...
- *Networking:* wfq, wf2q, sfq, drr, csfq, ...
- *Datacenters:* Hadoop's fair sched, Dominant Resource Fairness (DRF)

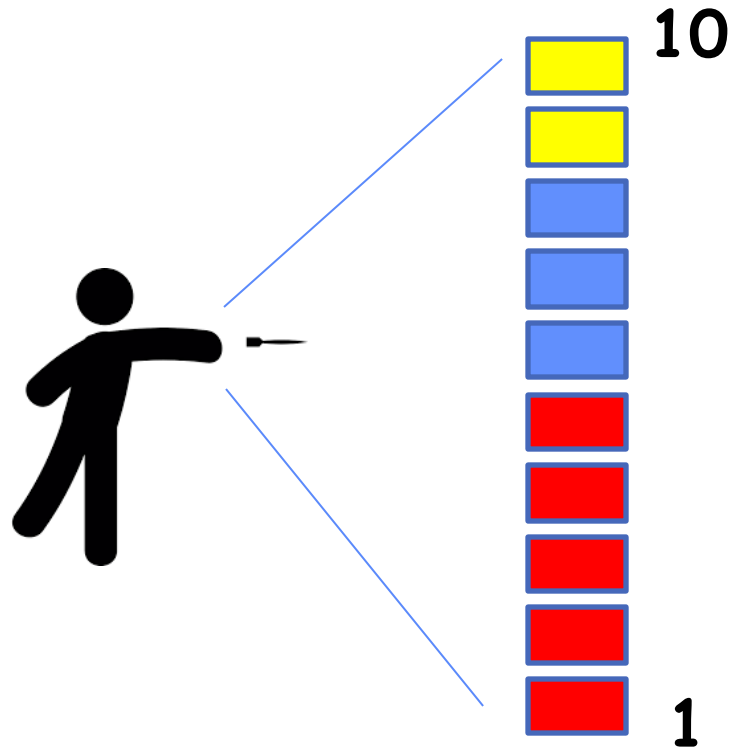
Lottery Scheduling

An approximation of weighted fair sharing

- Weight $\rightarrow$ number of tickets
- Scheduling decision $\rightarrow$ probabilistic: give a slot to a process proportionally to its weight



Lottery Scheduling: Simple Mechanism



- $N_{ticket} = \sum N_i$
- Pick a number d in $1 \dots N_{ticket}$ as the random “dart”
- Jobs record their N_i of allocated tickets
- Order them by N_i
- Select the first j such that $\sum N_i$ up to j exceeds d .

Lottery Scheduling Example (Cont.)

- Lottery Scheduling Example
 - Assume short jobs get 10 tickets, long jobs get 1 ticket

# short jobs/ # long jobs	% of CPU each short jobs gets	% of CPU each long jobs gets
1/1	91%	9%
0/2	N/A	50%
2/0	50%	N/A
10/1	9.9%	0.99%
1/10	50%	5%

- What if too many short jobs to give reasonable response time?
 - » If load average is 100, hard to make progress
 - » One approach: log some user out

Fair Queueing: Max-min Fairness originated in Networking

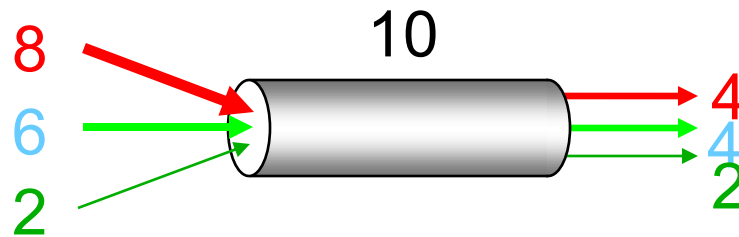
- Fair queueing explained in a **fluid flow system**: link (resource) allocated in arbitrary small quanta
 - Each flow receives $\min(r_i, f)$, where
 - » r_i – flow arrival rate
 - » f – link fair rate (see next slide)
- Weighted Fair Queueing (WFQ) – associate a weight with each flow [Demers, Keshav & Shenker '89]
 - In networking, a fluid flow system reduces to bit-by-bit round robin
- WFQ in a fluid flow system → Generalized Processor Sharing (GPS) [Parekh & Gallager '92]

Fair Rate Computation

- If link congested, compute f such that

This is the same value as
computed by max-min fair
algorithm

$$\forall_i \min(r_i, f) = C$$

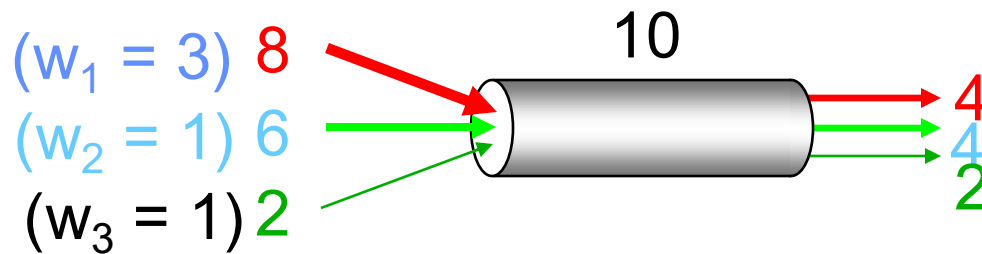


$f = 4$:
 $\min(8, 4) = 4$
 $\min(6, 4) = 4$
 $\min(2, 4) = 2$

Fair Rate Computation

- Associate a weight w_i with each flow i
- If link congested, compute f such that

$$\sum_i \min(r_i, f w_i) = C$$



$f = 2$:

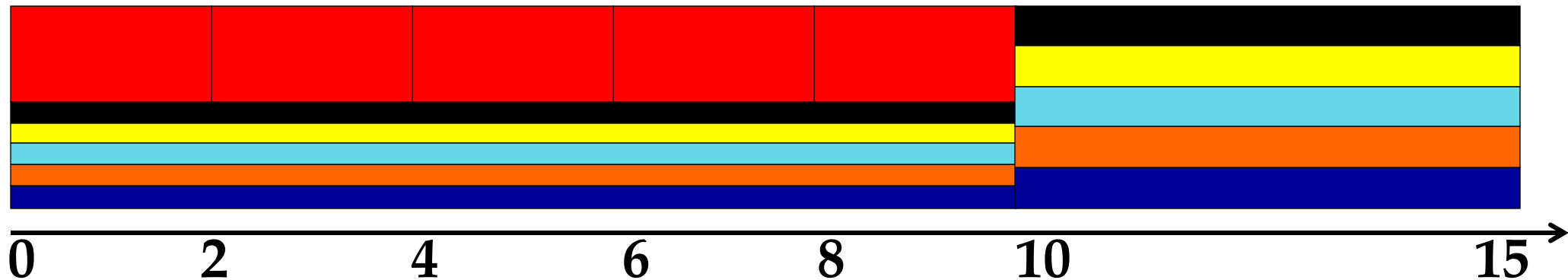
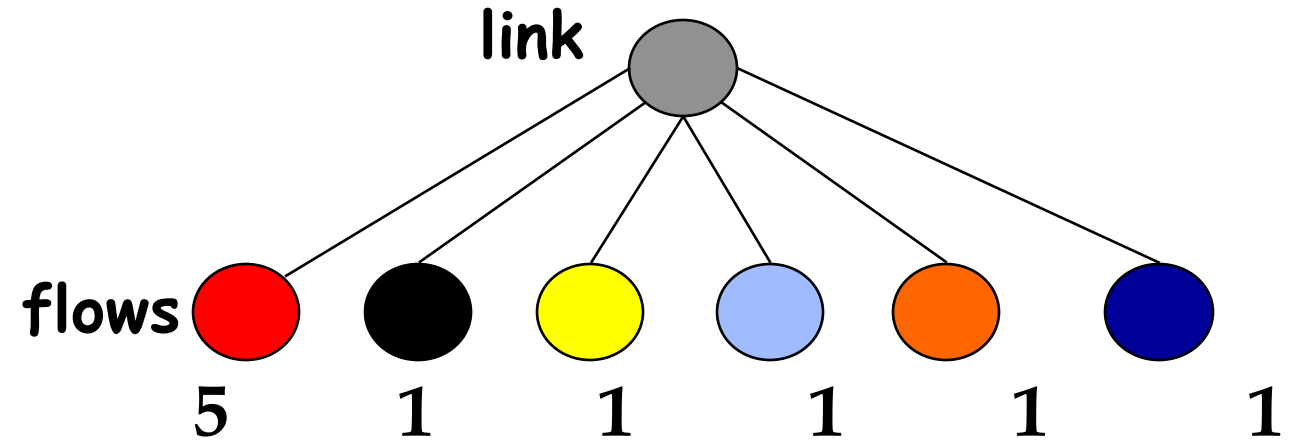
$$\min(8, 2 \cdot 3) = 6$$
$$\min(6, 2 \cdot 1) = 2$$
$$\min(2, 2 \cdot 1) = 2$$

Fluid Flow System

- Flows can be served one bit at a time
 - Fluid flow system, also known as Generalized Processor Sharing (GPS) [Parekh and Gallager '93]
- WFQ can be implemented using **bit-by-bit weighted round robin** in GPS model
 - During each round from each flow that has data to send, send a number of bits equal to the flow's weight

Generalized Processor Sharing Example

- **Red session** has packets backlogged between 0 and 10
 - Other sessions have packets continuously backlogged
- Each packet has size 1
- Link capacity is 1

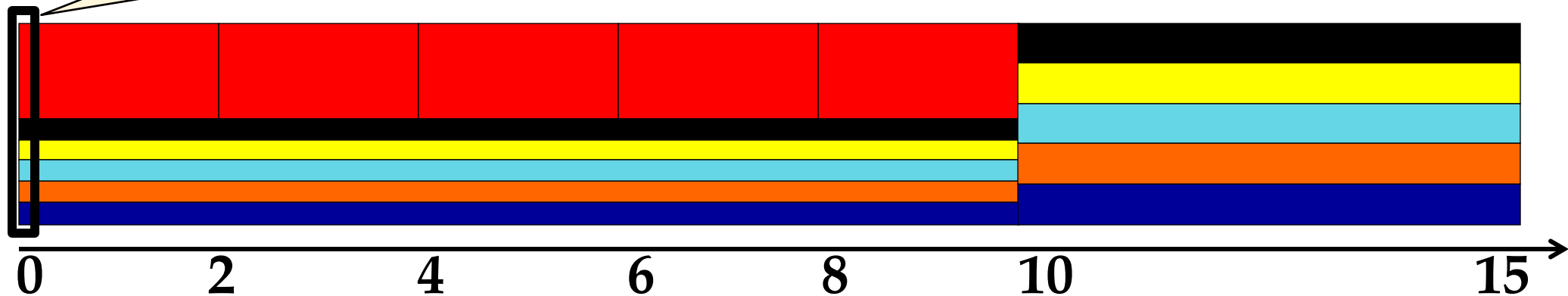
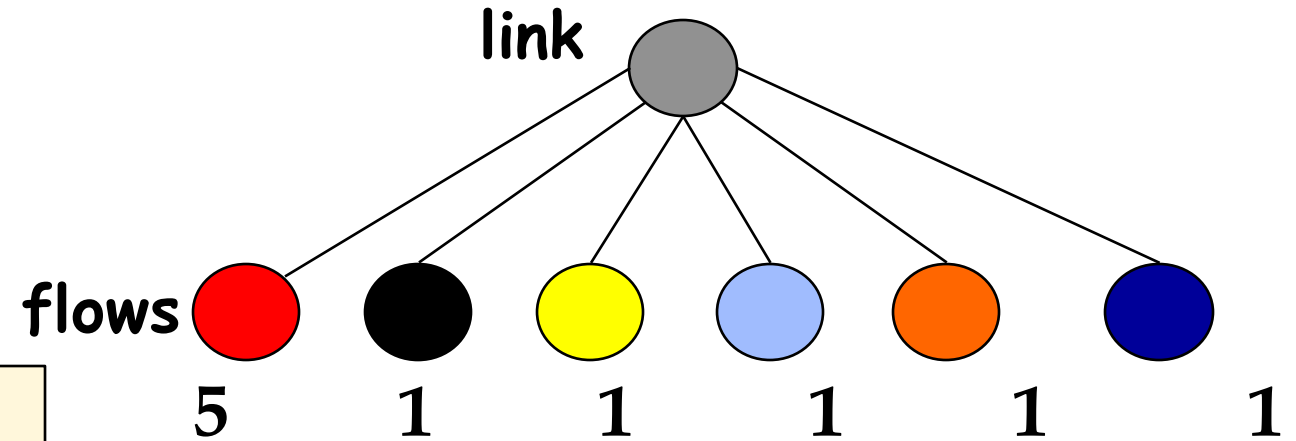


Generalized Processor Sharing Example

- **Red session** has packets backlogged between 0 and 10
 - Other sessions have packets continuously backlogged

- Each packet has size 1

- Link Each vertical column is a round, in our case, 10 bits (5, 1, 1, 1, 1, 1)

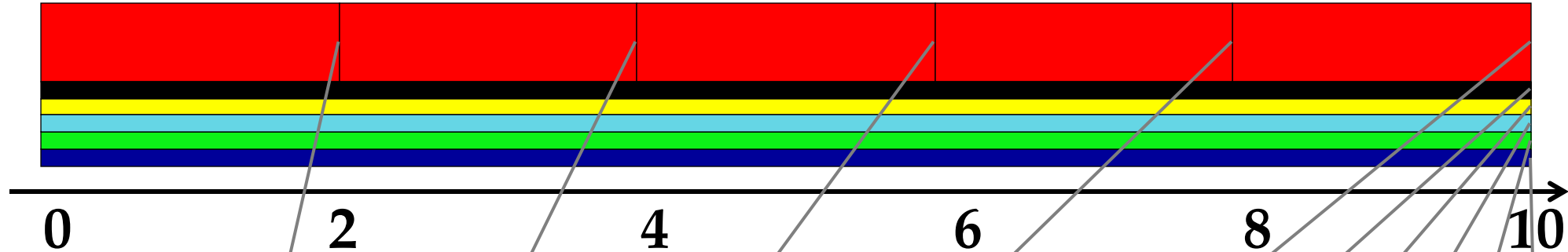


Packet Approximation of GPS

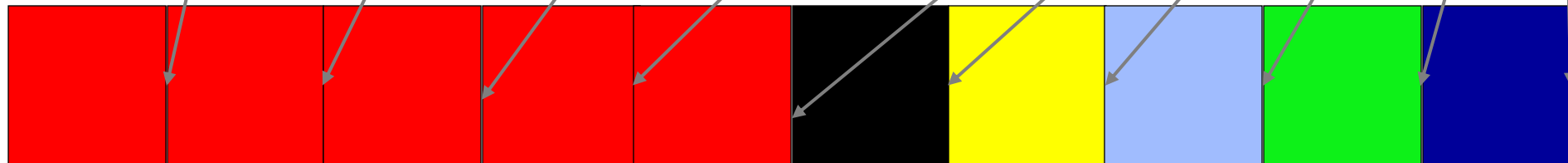
- Emulate GPS
- Select packet that finishes first in GPS assuming that there are no future arrivals

Approximating GPS with WFQ

- Fluid GPS system service order



- Weighted Fair Queueing
 - select the first packet that finishes in GPS



Administrivia

- Midterm I:
 - Delayed grade release to end of the week (Sorry)
 - Solutions will be up off the Resources page
- One thing to remember: **you learn by doing!**
 - Don't use chatgpt to do your homework as you won't learn much!
- Next lecture:
 - Both Matei and me will out (I'll be at SOSPP in Korea)
 - The lecture will be given by Yifan Qiao
 - System postdoc in Sky Computing Lab



Yifan Qiao



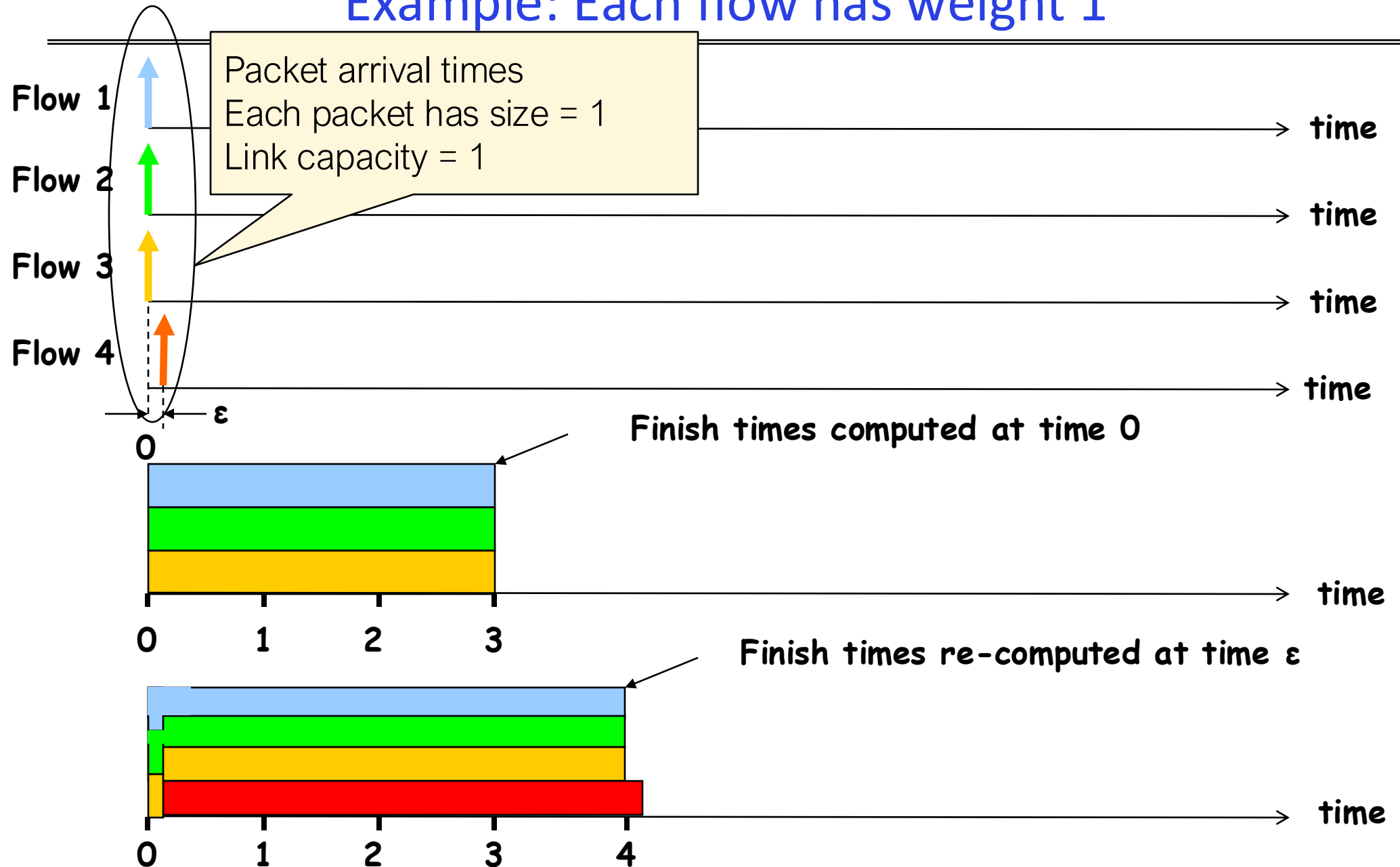
Postdoc at [University of California, Berkeley](#)
Verified email at berkeley.edu - [Homepage](#)
[Operating Systems](#) [Cloud Computing](#) [ML Systems](#)

TITLE	CITED BY	YEAR
Dorylus: Affordable, Scalable, and Accurate GNN Training with Distributed CPU Servers and Serverless Threads J Thorpe, Y Qiao, J Eyoifson, S Teng, G Hu, Z Jia, J Wei, K Vora, ... 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI) ...	198	2021
Bamboo: Making Preemptible Instances Resilient for Affordable Training of Large DNNs J Thorpe, P Zhao, J Eyoifson, Y Qiao, Z Jia, M Zhang, R Netravali, GH Xu 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI) ...	97	2023
MemLiner: Lining up Tracing and Application for a Far-Memory-Friendly Runtime C Wang, H Ma, S Liu, Y Qiao, J Eyoifson, C Navasca, S Lu, GH Xu 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI) ...	55	2022
Canvas: Isolated and Adaptive Swapping for Multi-Applications on Remote Memory C Wang, Y Qiao, H Ma, S Liu, W Chen, R Netravali, M Kim, GH Xu 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI) ...	54	2023
Hermit: Low-Latency, High-Throughput, and Transparent Remote Memory via Feedback-Directed Asynchrony Y Qiao, C Wang, Z Ruan, A Belay, Q Lu, Y Zhang, M Kim, GH Xu 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI) ...	46	2023
Mako: A Low-Pause, High-Throughput Evacuating Collector for Memory-Disaggregated Datacenters	27	2022

Implementation Challenge

- Need to compute the finish time of a packet in the fluid flow system...
- ... but the finish time may change as new packets arrive!
- Need to update the finish times of all packets that are in service in the fluid flow system when a new packet arrives
 - But this is very expensive; a high-speed router may need to handle hundred of thousands of flows!

Example: Each flow has weight 1

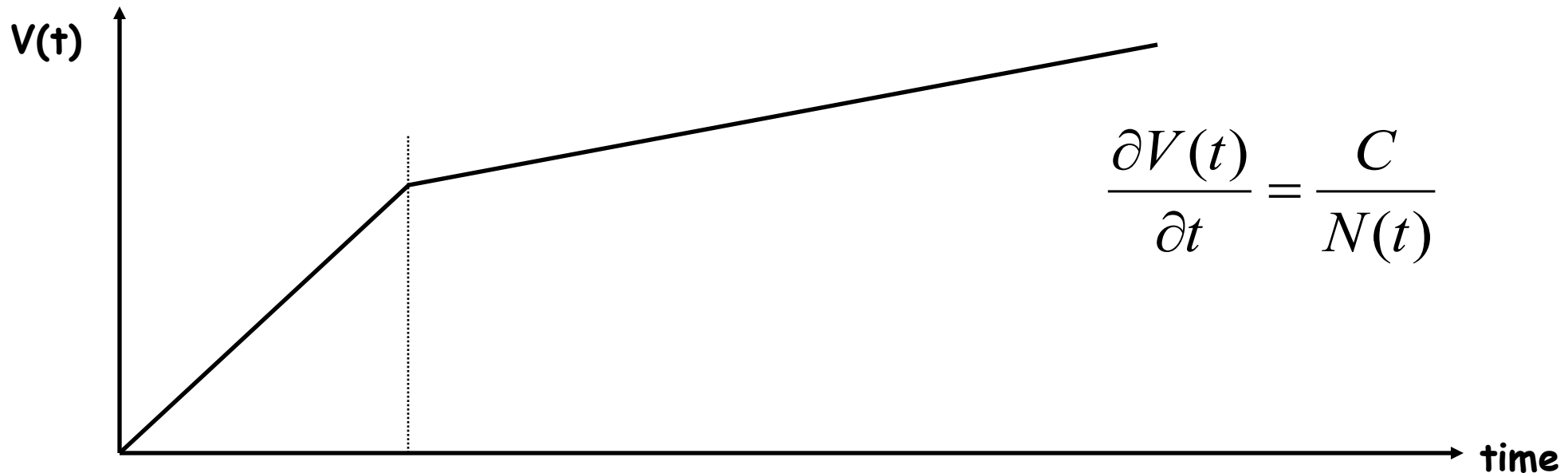


Solution: Virtual Time

- **Key Observation:** while the finish times of packets may change when a new packet arrives...
 - The order in which packets finish doesn't!
 - Only the order is important for scheduling
- **Solution:** instead of the packet finish time maintain the number of rounds needed to send the remaining bits of the packet (**virtual finishing time**)
 - Virtual finishing time doesn't change when the packet arrives
- **System virtual time** – **index of the round in the bit-by-bit round robin scheme**

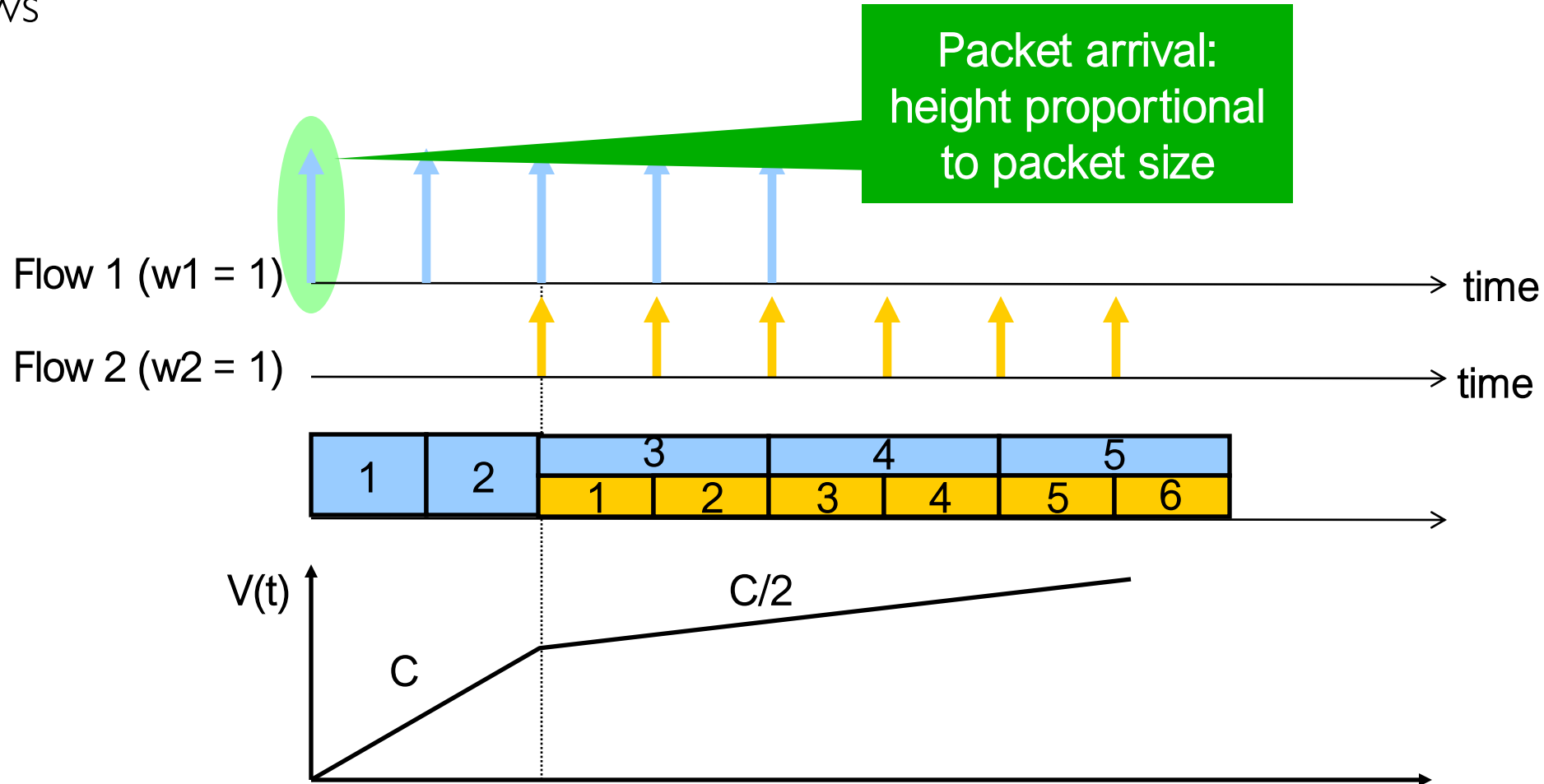
System Virtual Time: $V(t)$

- Measure service, instead of time
- $V(t)$ slope – normalized rate at which every backlogged flow receives service in the fluid flow system
 - C = link capacity
 - $N(t)$ = total weight of backlogged flows in fluid flow system at time t



System Virtual Time ($V(t)$): Example

- $V(t)$ increases inversely proportionally to the sum of the weights of the backlogged flows



Fair Queueing Implementation

- Define
 - F_i^k virtual finishing time of packet k of flow i
 - a_i^k arrival time of packet k of flow i
 - L_i^k length of packet k of flow i
 - w_i weight of flow i
- The finishing time of packet $k+1$ of flow i is

$$F_i^{k+1} = \max(V(a_i^{k+1}), F_i^k) + L_i^{k+1} / w_i$$

Round by which
packet $k+1$ is
served

Current
round

Round when last
packet of flow i
finishes

of rounds it takes
to serve new
packet (i.e., packet
 $k+1$ of flow i)

Early Eligible Virtual Deadline First (EEVDF)

EEVDF Scheduler

English

The “Earliest Eligible Virtual Deadline First” (EEVDF) was first introduced in a scientific publication in 1995 [1].

The Linux kernel began transitioning to EEVDF in version 6.6 (as a new option in 2024), moving away from the earlier Completely Fair Scheduler (CFS) in favor of a version of EEVDF proposed by Peter Zijlstra in 2023 [2-4]. More information regarding CFS can be found in [CFS Scheduler](#).

A Proportional Share Resource Allocation Algorithm for Real-Time, Time-Shared Systems

Ion Stoica^{*} Hussein Abdel-Wahab[†] Kevin Jeffay[‡] Sanjoy K. Baruah[§]
Johannes E. Gehrke[¶] C. Greg Plaxton^{||}

Abstract

We propose and analyze a proportional share resource allocation algorithm for realizing real-time performance in time-shared operating systems. Processes are assigned a weight which determines a share (percentage) of the resource they are to receive. The resource is then allocated in discrete-sized time quanta in such a manner that each process makes progress at a precise, uniform rate. Proportional share allocation algorithms are of interest because (1) they provide a

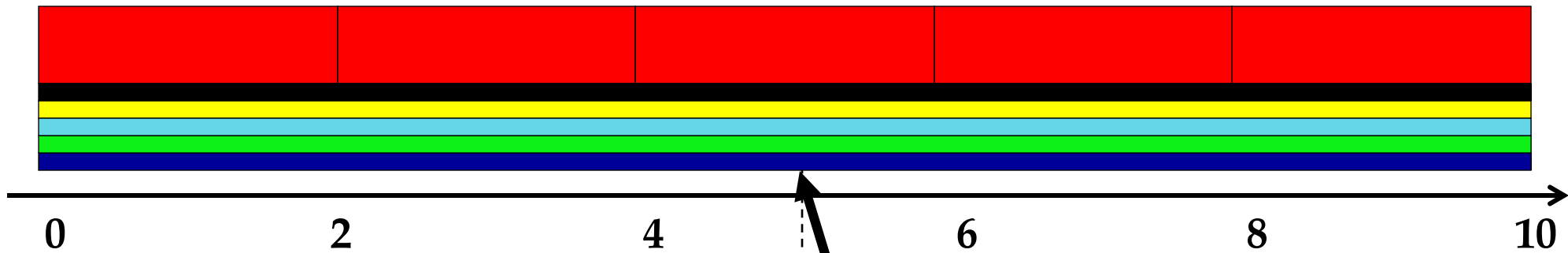
time quantum. In addition, the algorithm provides support for dynamic operations, such as processes joining or leaving the competition, and for both fractional and non-uniform time quanta. As a proof of concept we have implemented a prototype of a CPU scheduler under FreeBSD. The experimental results shows that our implementation performs within the theoretical bounds and hence supports real-time execution in a general purpose operating system.

<https://www.kernel.org/doc/html/next/scheduler/sched-eevdf.html>

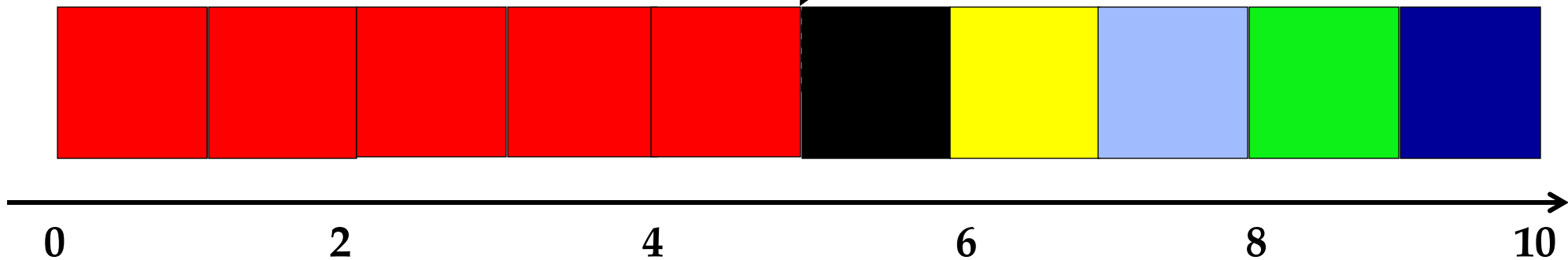
What problem does EEVDF try to solve?

Minimize lag: the difference between service received in real system vs fluid flow (idealized) system

- Fluid system service order



- Weighted Fair Queueing



Fluid system: red flow got 2.5

Real system: red flow got 5

Lag: -2.5 (worst case $O(n)$
where n is the number of
processes/flows)

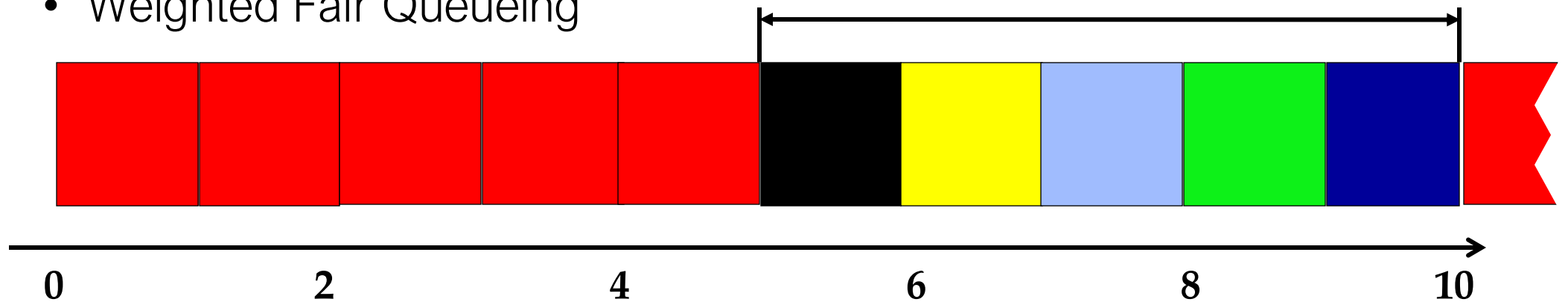
Why is this bad?

Minimize lag: the difference between service received in real system vs fluid flow (idealized) system

- Fluid system service order



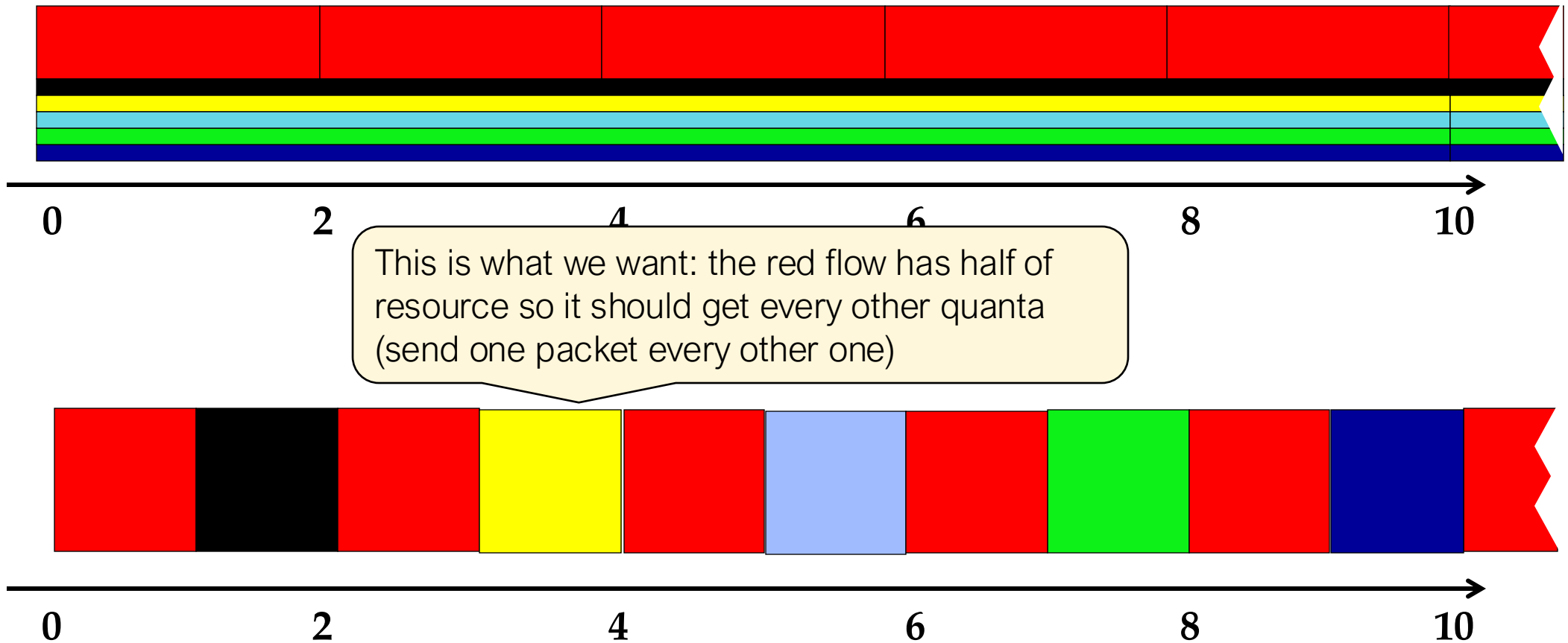
- Weighted Fair Queueing



We want this!

Minimize lag: the difference between service received in real system vs fluid flow (idealized) system

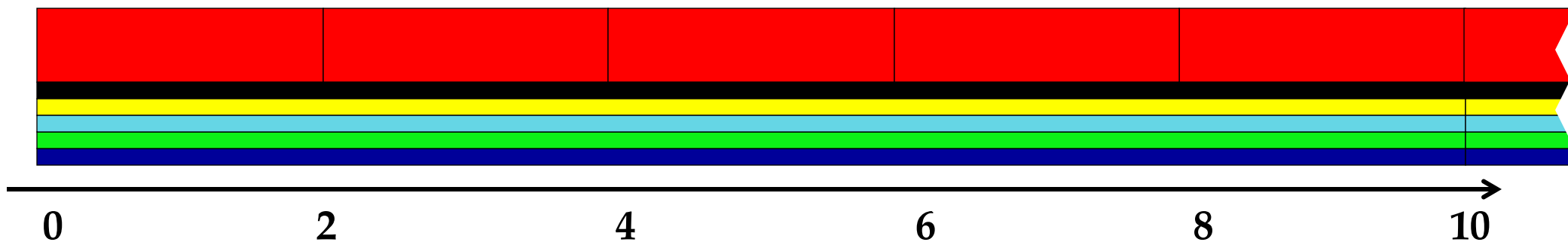
- Fluid system service order



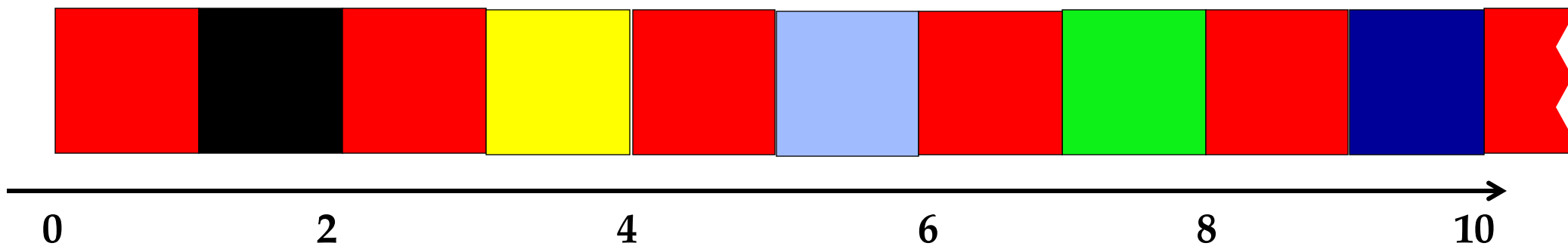
We want this!

Minimize lag: the difference between service received in real system vs fluid flow (idealized) system

- Fluid system service order



Lag ≤ 0.5 (independent on number of flows)!



How?

Schedule only the processes/packets that are eligible in fluid flow system, i.e., packets with virtual starting time greater than virtual time

- Fluid system service order



Only first of the red packets is eligible and has earliest deadline among all eligible packets so schedule it

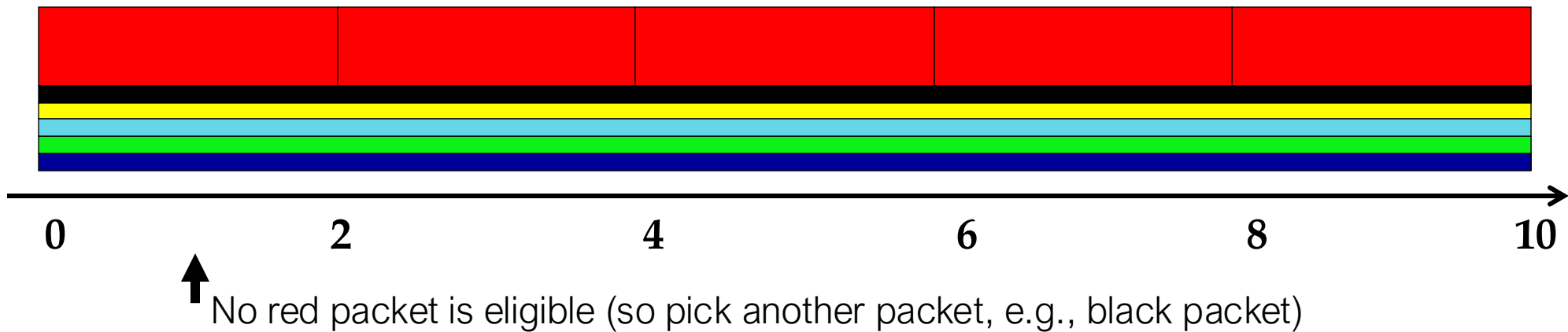
- Weighted Fair Queueing



How?

Schedule only the processes/packets that are eligible in fluid flow system, i.e., packets with virtual arrival time greater than virtual time

- Fluid system service order



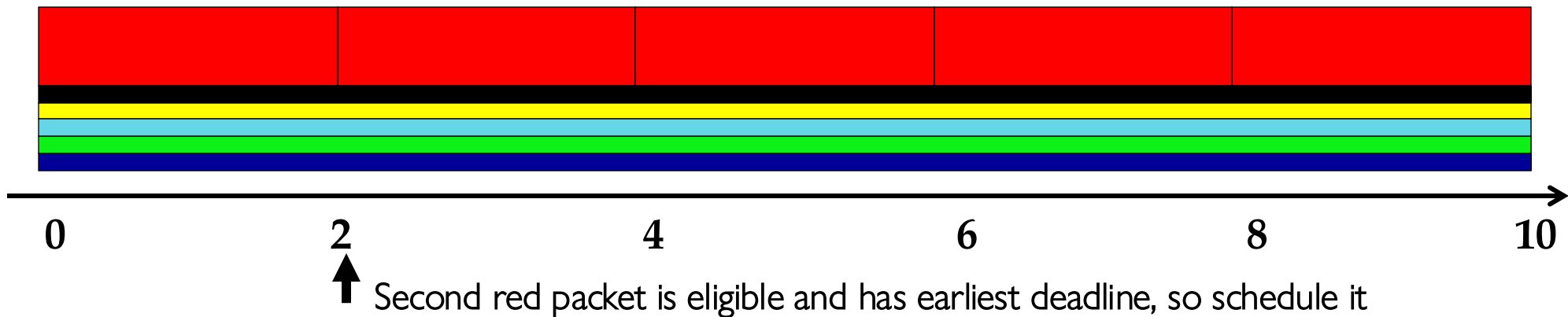
- Weighted Fair Queueing



How?

Schedule only the processes/packets that are eligible in fluid flow system, i.e., packets with virtual arrival time greater than virtual time

- Fluid system service order



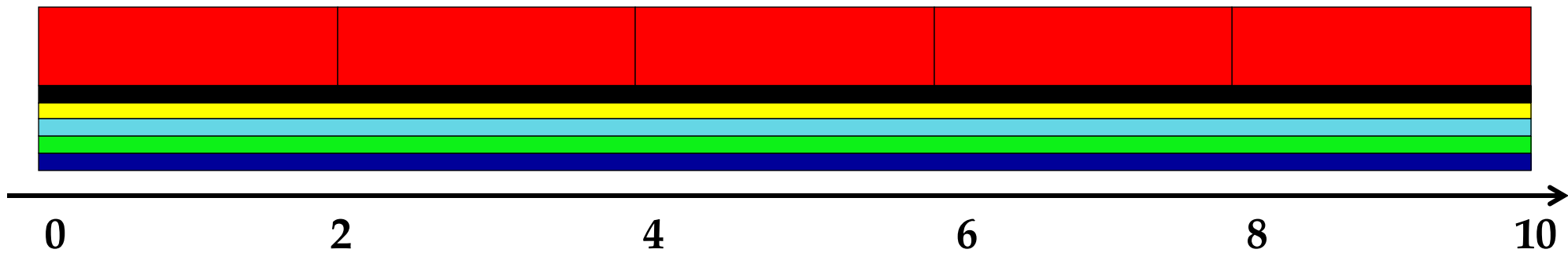
- Weighted Fair Queueing



How?

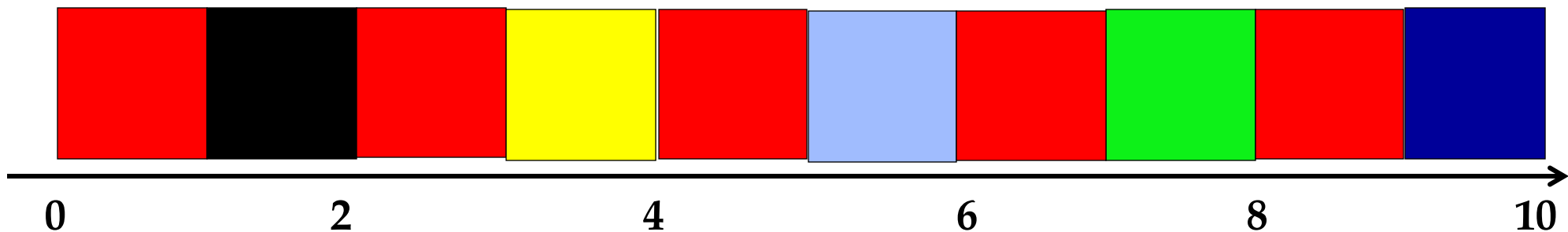
Schedule only the processes/packets that are eligible in fluid flow system, i.e., packets with virtual arrival time greater than virtual time

- Fluid system service order



Lag ≤ 0.5 (independent on number of flows)

- Weighted Fair Queueing



Comparison of proportional share disciplines

- n : number of flows / processes / users
- q : length of max time quanta

	Missing the deadline in fluid flow system	Service lag to fluid flow systems	Scheduling complexity	Comments
Fair sharing / queueing	q	$O(n)*q$	$O(\log(n))$	- For networking q is max packet size - Data structure: priority queue ordered by finishing virtual time
EEVDF	q	$O(1)*q$	$O(\log(n))$	More complex data structure
Lottery scheduling	$O(\sqrt{n})*q$	$O(\sqrt{n})*q$	$O(1)$	
Round robin	q	$O(n)*q$	$O(n)$	- If only one flow has packets to send, it takes $O(n)$ to check every empty queue - Hard to handle diff. size time quanta

Conclusion

Max-min fairness

- Each job gets its “fair share”
- Provides isolation: a job is guaranteed its fair share

Weighted fair sharing:

- A way to provide priorities: give higher weights to higher priority jobs
- Avoid starvation: as long as a job has a non-negative weight it will get service

Various implementations trading between complexity, accuracy, efficiency:

- Fair Sharing, EEVDF, Lottery Scheduling